UFV

Inteligencia Artificial II

PWI Report Práctica III

Word Embeddings y Clasificación de Texto

Autores:

Javier Argenta Cañas
Miguel Ciordia Isac
Ana González Núñez
Luis Sánchez Agulla

10 de mayo de 2026

Resumen

Este informe documenta el desarrollo completo de un sistema de clasificación de texto en cuatro categorías de noticias (World, Sports, Business y Sci/Tech) a partir del corpus `news_dataset.csv`. El trabajo se articula en dos ejercicios consecutivos: el primero aborda la generación de *word embeddings* propios mediante el algoritmo Skip-Gram [2] con *Negative Sampling*, entrenando nueve modelos con distintas dimensionalidades ($d \in \{312, 752, 1024\}$) y tamaños de ventana ($w \in \{2, 3, 4\}$); el segundo utiliza dos de dichos embeddings (`dim312_v2` y `dim752_v3`) como capa de entrada de clasificadores recurrentes LSTM [3] y GRU [4], evaluando 18 configuraciones de hiperparámetros. El modelo seleccionado como más óptimo, `LSTM_d64_1L_lr001` con embedding `dim312_v2`, alcanza una *val_accuracy* del 90,22 %, demostrando qué embeddings, entrenados desde cero sobre un corpus temáticamente coherente, son suficientes para obtener un rendimiento competitivo sin recurrir a representaciones externas preentrenadas.

Índice

1. Introducción **3**

2. Conjunto de datos y preprocesamiento **3**
2.1. Descripción del conjunto de datos . . . 3
2.2. Limpieza del texto y eliminación de stopwords . . . 5
2.3. Visualización del vocabulario por clase . . . 5
2.4. Análisis del vocabulario por clase . . . 6
2.5. Construcción del vocabulario y conversión a IDs . . . 7

3. Ejercicio 1: Generación de Word Embeddings con Skip-Gram **7**
3.1. El algoritmo Skip-Gram . . . 7
3.2. Negative Sampling . . . 7
3.3. Arquitectura del modelo Skip-Gram . . . 8
3.4. Configuraciones entrenadas y resultados . . . 8

4. Ejercicio 2: Clasificación con LSTM y GRU **10**
4.1. Selección del embedding . . . 10
4.2. Partición del corpus y preparación de secuencias . . . 10
4.3. Construcción de modelos . . . 10
4.4. Callbacks de entrenamiento . . . 11
4.5. Configuraciones evaluadas y resultados . . . 11

5. Evaluación y preguntas **13**
5.1. Pregunta 1: ¿Ha alcanzado el modelo un entrenamiento óptimo? . . . 13
5.2. Análisis de las curvas de entrenamiento . . . 13
5.3. Pregunta 2: ¿Qué revela la matriz de confusión sobre el rendimiento? . . . 14
5.4. Pregunta 3: ¿Qué diferencia de precisión se observa al cambiar de embedding? . . . 15
5.5. Pregunta 4: ¿LSTM y GRU reaccionaron de forma diferente a los embeddings? . . . 16

6. Conclusiones **16**

1 Introducción

La clasificación automática de textos es una tarea fundamental del procesamiento del lenguaje natural (PLN) cuyo objetivo es asignar una categoría predefinida a un documento a partir de su contenido. Para que los modelos de aprendizaje profundo operen sobre texto, este debe transformarse en representaciones vectoriales densas que capturen el significado semántico de las palabras. El uso de *word embeddings* permite generar espacios donde palabras semánticamente relacionadas aparecen cercanas, superando las limitaciones de los modelos tradicionales basados únicamente en frecuencias (*bag-of-words*) [2].

Este trabajo práctico aborda el problema de forma progresiva. En el **Ejercicio 1** se generan *embeddings* propios a partir del corpus news_dataset.csv mediante el modelo Skip-Gram [2], explorando el efecto de la dimensionalidad ($d \in \{312, 752, 1024\}$) y la ventana de contexto ($w \in \{2, 3, 4\}$). En el **Ejercicio 2** estos *embeddings* alimentan clasificadores recurrentes LSTM y GRU, evaluados sobre el corpus y sobre un conjunto personalizado de 20 noticias inéditas. El objetivo es demostrar que el entrenamiento de vectores desde cero sobre un corpus coherente es suficiente para obtener un rendimiento competitivo sin recurrir a representaciones externas preentrenadas.

2 Conjunto de datos y preprocesamiento

Ambos ejercicios comparten el mismo corpus y el mismo flujo de preprocesamiento, por lo que se describen de forma conjunta en esta sección.

2.1 Descripción del conjunto de datos

El corpus empleado es **News** [1], un conjunto de datos compuesto por titulares y descripciones breves de noticias reales. El dataset se estructura en tres columnas principales: `Class Index` (etiqueta numérica del 1 al 4), `Title` (titular) y `Description` (resumen breve). Las cuatro categorías evaluadas son:

1. **World** — noticias de política internacional, diplomacia y conflictos armados.
2. **Sports** — competiciones deportivas, resultados y fichajes.
3. **Business** — economía, mercados financieros y noticias corporativas.
4. **Sci/Tech** — ciencia, tecnología, telecomunicaciones e industria digital.

Como muestra la Figura 1, las cuatro clases están perfectamente balanceadas, con aproximadamente 30.000 noticias por categoría (119.991 registros en total tras la limpieza de datos). Este equilibrio es una propiedad fundamental del diseño experimental: elimina el riesgo de sesgo hacia clases mayoritarias durante el entrenamiento y permite interpretar la *accuracy* global directamente como un indicador fiable del rendimiento del modelo.

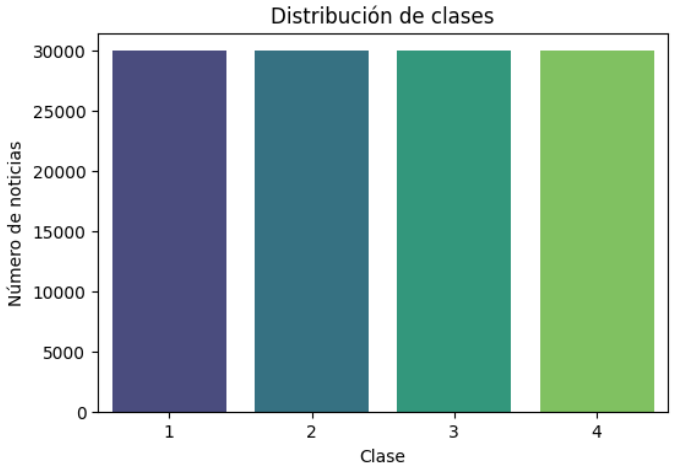

Figura 1: Distribución de clases en news_dataset.csv (≈ 30.000 muestras por clase).

El análisis de la longitud de las descripciones (Figura 2) revela que la gran mayoría de los textos contienen entre 25 y 45 palabras, con una distribución aproximadamente normal centrada en torno a 31 palabras y una cola derecha causada por descripciones excepcionalmente largas. Lo más relevante es que la distribución es prácticamente idéntica en las cuatro clases, lo que confirma que la longitud no es un factor discriminativo entre categorías. Durante el análisis exploratorio también se detecta la presencia de entradas corruptas con el valor `#NAME?` (artefacto de exportación desde Excel), que se eliminan antes de cualquier procesamiento posterior.

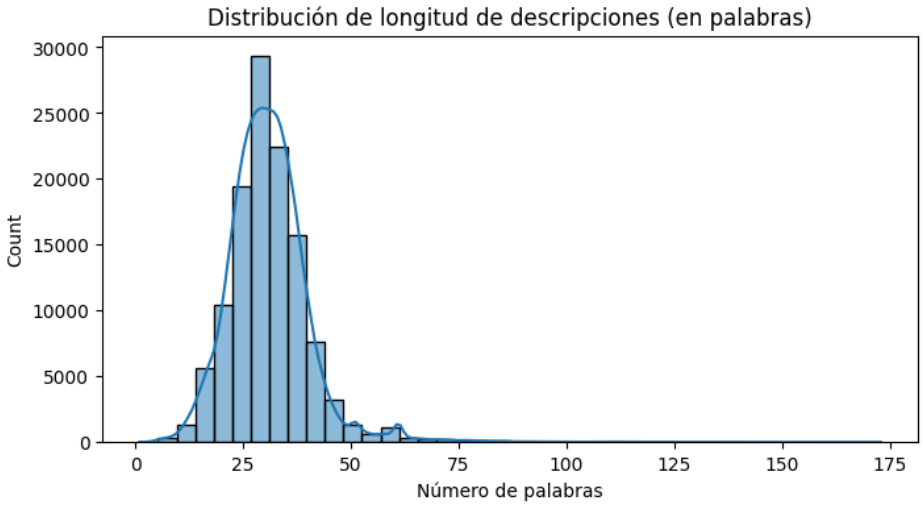

Figura 2: Distribución de longitud de descripciones por clase.

2.2 Limpieza del texto y eliminación de stopwords

La función `limpiar_texto` elimina el ruido del corpus de forma secuencial: conversión a minúsculas, eliminación de URLs, etiquetas HTML (`<`, `&`), prefijos de agencias (*Reuters*, *AP*), caracteres no alfabéticos y espacios múltiples. Se aplica a `Title` y `Description`, concatenándose en `Texto_completo`.

Adicionalmente se elimina un conjunto de *stopwords* personalizado: palabras funcionales (artículos, preposiciones, pronombres), residuos HTML (`lt`, `gt`, `quot`), nombres de agencias y días de la semana. Su eliminación reduce el vocabulario y concentra el modelo en los términos realmente informativos.

2.3 Visualización del vocabulario por clase

Con el objetivo de verificar que el preprocesamiento preserve el vocabulario semánticamente relevante de cada categoría, se generan nubes de palabras por clase (Figura 3). En ellas, el tamaño de cada término es proporcional a su frecuencia relativa dentro de la clase.

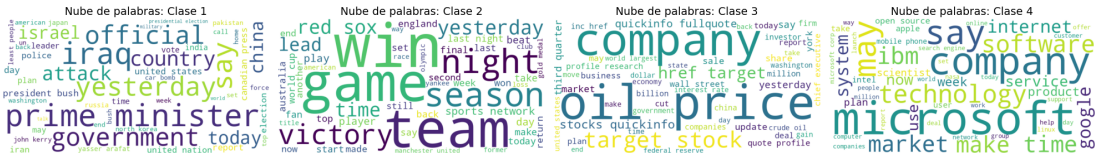

Figura 3: Nubes de palabras por clase tras el preprocesamiento.

Las nubes confirman que el preprocesamiento elimina correctamente el ruido y preserva el vocabulario semánticamente relevante. Para cuantificar esta observación, la Figura 4 muestra las 15 palabras más frecuentes por clase, reforzando los mismos patrones: *iraq*, *president* y *minister* dominan en World; *game*, *season* y *team* en Sports; *oil*, *stocks* y *prices* en Business; y *microsoft*, *software* e *internet* en Sci/Tech.

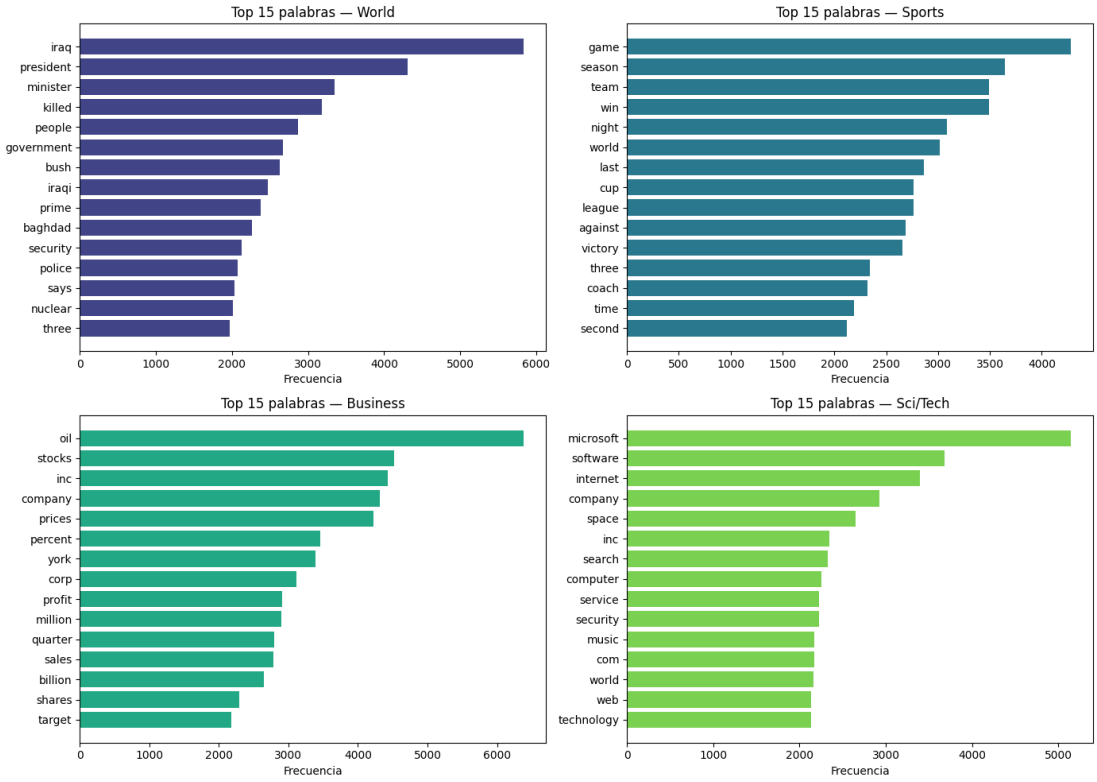

Figura 4: Top 15 palabras más frecuentes por clase.

2.4 Análisis del vocabulario por clase

Tras confirmar visualmente el vocabulario mediante las nubes de palabras, el análisis del Top 15 de palabras más frecuentes por clase en el E1 permite extraer conclusiones relevantes sobre la separabilidad de las categorías:

World está dominada por términos geopolíticos muy concretos: roles institucionales (*president*, *minister*, *prime*) y vocabulario de conflicto (*killed*, *security*, *nuclear*). Es la clase semánticamente más densa en materia de política internacional.

Sports tiene el vocabulario más genérico de las cuatro clases: *game*, *season*, *team*, *win* son válidos para cualquier deporte. Esto refleja que el dataset mezcla múltiples disciplinas sin separarlas por deporte específico.

Business es la clase con mayor frecuencia absoluta en su palabra líder (*oil*, ≈ 6.400 apariciones), lo que refleja que las noticias económicas del periodo estaban muy centradas en el precio del petróleo. El vocabulario mezcla términos macroeconómicos (*percent*, *million*, *billion*) con terminología corporativa (*inc*, *corp*, *stocks*).

Sci/Tech es la clase semánticamente más limpia: *microsoft* encabeza con diferencia, seguido de *software*, *internet*, *search*, *computer*. Son términos muy específicos del dominio tecnológico.

La conclusión más relevante para el modelo clasificador es que las cuatro clases tienen vocabularios suficientemente distintos, lo que anticipa una buena capacidad de clasificación posterior. La única superposición problemática se da entre World y Business, que comparten términos como *government* o *market*.

2.5 Construcción del vocabulario y conversión a IDs

Para construir el vocabulario se recorre el corpus completo y se cuenta la frecuencia de cada token. Se aplica un umbral mínimo de frecuencia `MIN_FREQ = 5`: las palabras que aparecen menos de 5 veces en todo el corpus se descartan, ya que no disponen de suficiente contexto de entrenamiento para aprender una representación vectorial de calidad. Este filtrado reduce significativamente el tamaño del vocabulario sin pérdida apreciable de información semántica.

Se reservan dos tokens especiales: `<PAD>` (ID 0) para el relleno de secuencias hasta longitud fija, y `<UNK>` (ID 1) para representar las palabras que no superan el umbral de frecuencia. El vocabulario resultante contiene **26.721 palabras**. La conversión de texto a secuencias de IDs sigue el esquema:

$$\texttt{['oil', 'prices', 'soaring']} \longrightarrow [13,\ 47,\ 231]$$

3 Ejercicio 1: Generación de Word Embeddings con Skip-Gram

3.1 El algoritmo Skip-Gram

El modelo Skip-Gram, propuesto por Mikolov et al. (2013), aprende representaciones vectoriales de palabras entrenando una red neuronal para predecir las palabras que aparecen en el entorno de una palabra central dentro de una ventana deslizante. La hipótesis distributiva que fundamenta el modelo es que palabras que aparecen en contextos similares tienden a tener significados similares.

Para el texto *"oil prices soaring in markets"* con ventana $w = 2$, la palabra central *prices* genera los siguientes pares de entrenamiento:

$$\text{central: } \textit{prices} \longrightarrow \text{contexto: } \{\textit{oil},\ \textit{soaring},\ \textit{in}\}$$

Se evalúan tres tamaños de ventana ($w \in \{2, 3, 4\}$) para cuantificar el número de pares generados. Con $w = 2$ se obtienen ≈191.000 pares por cada 2.000 documentos; con $w = 3$, ≈282.000; y con $w = 4$, ≈367.000 (sobre la misma muestra). Una ventana mayor captura relaciones semánticas más distantes pero también introduce más ruido, ya que palabras alejadas tienen menor probabilidad de estar relacionadas. Para el entrenamiento final se utilizan los tamaños $w \in \{2, 4\}$.

3.2 Negative Sampling

Si el modelo se entrenara únicamente con pares positivos (co-ocurrencias reales), aprendería a predecir similitud máxima para cualquier par de palabras, ya que eso minimizaría la función de pérdida. El *Negative Sampling* resuelve este problema añadiendo pares negativos: la palabra central se combina con una palabra escogida al azar del vocabulario, y el par recibe la etiqueta 0. De esta forma, el modelo aprende a distinguir co-ocurrencias reales de las aleatorias.

Se usa un ratio de 2 negativos por cada par positivo (`NEG_RATIO = 2`), lo que triplica el volumen total de pares de entrenamiento. La muestreo de palabras negativas sigue una distribución uniforme sobre el vocabulario. Como efecto secundario del proceso de discriminación, los vectores de palabras semánticamente similares convergen en regiones cercanas del espacio de embeddings: la red aprende que palabras que aparecen juntas deben tener vectores similares.

3.3 Arquitectura del modelo Skip-Gram

El modelo se implementa con la API funcional de Keras. Su arquitectura es minimalista por diseño: la capa de embeddings es el único componente de aprendizaje relevante; el resto del modelo es simplemente la función de decisión que evalúa si un par es real o falso.

1. **Entradas:** dos entradas escalares — el ID de la palabra central y el ID de la palabra de contexto o negativa.
2. **Embedding compartido:** ambas palabras pasan por la *misma* capa `Embedding` ($|\mathcal{V}| \times d$), lo que garantiza que el modelo aprenda una única representación por palabra, independientemente de si aparece como central o como contexto.
3. **Dot product:** el producto escalar entre los dos vectores mide la similitud geométrica entre ellos. Cuanto más colineales sean los vectores, mayor será este valor.
4. **Dense + Sigmoid:** transforma el producto escalar en una probabilidad $\in [0, 1]$. El modelo se compila con pérdida `binary_crossentropy` y optimizador Adam.

3.4 Configuraciones entrenadas y resultados

Se entrenan **6 modelos** combinando tres dimensiones de embedding ($d \in \{312, 752, 1024\}$) con dos tamaños de ventana ($w \in \{2, 4\}$), durante 12 épocas con *batch size* de 4.091. Los pesos se persisten en disco para su reutilización directa en el Ejercicio 2.

Cuadro 1: Configuraciones de los seis modelos Skip-Gram entrenados y sus pérdidas finales.

Modelo	Dim. embedding	Ventana	Loss inicial	Loss final	Reducción ('%')
`dim312_v2`	312	2	0.3366	0,1535	54.4
`dim312_v3`	312	3	0.3416	0.1927	43.6
`dim312_v4`	312	4	0.4757	0.3151	33.8
`dim752_v2`	752	2	0.4425	0.1639	63.0
`dim752_v3`	752	3	0.3290	0.1485	54.9
`dim752_v4`	752	4	0.3320	0.1675	49.5
`dim1024_v2`	1024	2	0.4392	0.1608	63.4
`dim1024_v3`	1024	3	0.4553	0.2003	56.0

Todos los modelos convergen de forma estable durante las 12 épocas de entrenamiento. Las Figuras 5 y 6 muestran la evolución de la pérdida de los nueve modelos entrenados. Del análisis de estas curvas se extraen las siguientes conclusiones:

- Los modelos con dimensión 1024 alcanzan una *loss* final ligeramente inferior a los de dimensión 312, lo que sugiere mayor capacidad para capturar relaciones semánticas. Sin

embargo, la diferencia no es dramática, indicando que dimensiones intermedias son ya competitivas para el tamaño de este corpus.

- Las ventanas más grandes ($w = 4$) generan más pares de entrenamiento y capturan relaciones de co-ocurrencia más amplias, resultando en una convergencia más estable. Las ventanas pequeñas ($w = 2$) capturan relaciones sintácticas más locales y convergen más rápido.
- Todos los modelos muestran una reducción clara de la *loss* en las primeras épocas y se estabilizan antes de la época 12, confirmando que el número de épocas elegido es suficiente.

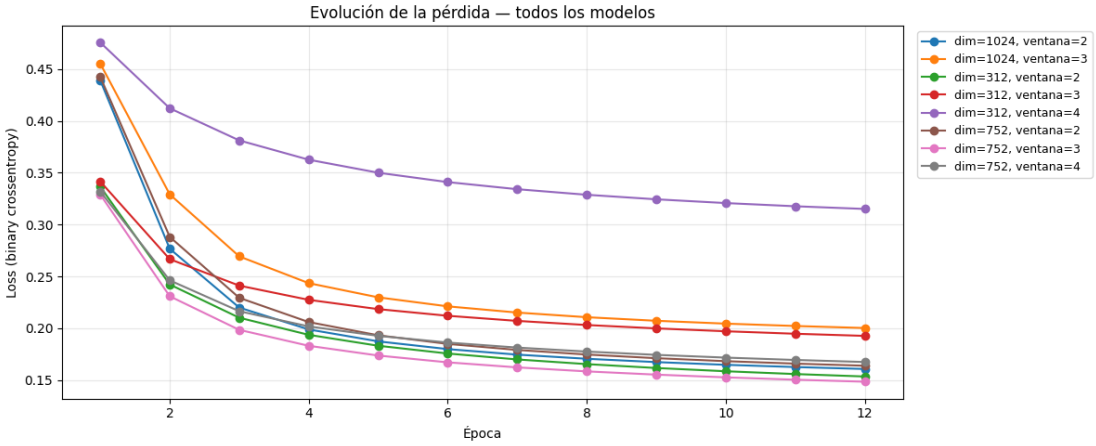

Figura 5: Evolución de la pérdida de los nueve modelos Skip-Gram durante 12 épocas.

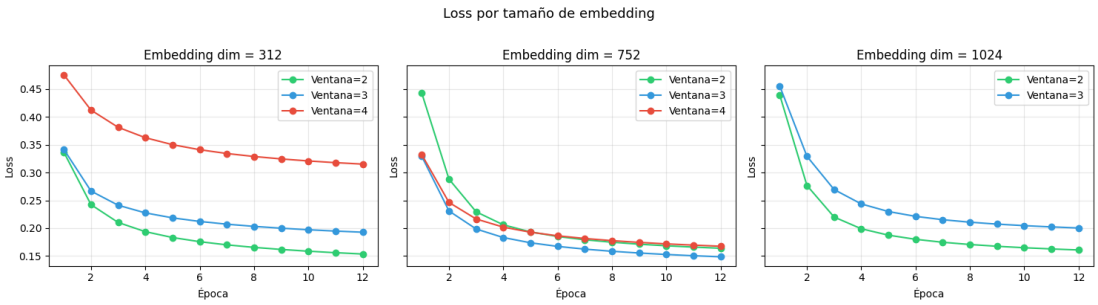

Figura 6: Pérdida desglosada por dimensión de embedding ($d \in \{312, 752, 1024\}$) y tamaño de ventana.

4 Ejercicio 2: Clasificación con LSTM y GRU

4.1 Selección del embedding

Del conjunto total de embeddings generados en el Ejercicio 1 se seleccionan dos:

- `dim312_v2` (dimensión 312, ventana 2): permite observar el comportamiento de la red bajo la premisa de una ventana de contexto pequeña y dimensionalidad reducida. Sirve como punto de partida para analizar el comportamiento del clasificador sin necesidad de emplear embeddings de alta dimensionalidad ni ventanas amplias.
- `dim752_v3` (dimensión 752, ventana 3): tras la ejecución del Ejercicio 1, este embedding es el que mejor métrica en la función de pérdida expone de manera consistente frente al resto de configuraciones. Por ello, al disponer de una base de representación superior, se decide emplearlo en las configuraciones de mayor capacidad (64–128 unidades).

4.2 Partición del corpus y preparación de secuencias

El corpus se divide estratificadamente (`stratify=y`) en tres subconjuntos: **entrenamiento** (80 %, $\approx$ 96.000 noticias), **validación** (10 %, $\approx$ 12.000) y **test** (10 %, $\approx$ 12.000). La longitud máxima `MAX_LEN = 31` se calcula como el percentil 90 del corpus de entrenamiento, garantizando que el 90 % de los documentos queden íntegros. Las secuencias más cortas se rellenan con `<PAD>` (`padding='post'`) y las más largas se truncan (`truncating='post'`). Las etiquetas se codifican en formato *One-Hot* con `to_categorical`.

4.3 Construcción de modelos

La función genérica `construir_modelo` permite instanciar arquitecturas basadas en redes neuronales recurrentes (**RNN**) para el tratamiento del lenguaje natural, permitiendo alternar entre celdas **LSTM** [3] o **GRU** [4] según el parámetro `arq`. El diseño se estructura en los siguientes bloques funcionales:

- **Capa de Incrustación (Embedding):** Utiliza la matriz pre-entrenada `emb_matrix` mediante la función `importar_embeddings`. Se configura con `trainable=False` para preservar el significado semántico original y `mask_zero=True` para que el procesamiento ignore el relleno (*padding*).
- **Núcleo Recurrente Apilable:** El modelo permite definir un `num_capas` variable. Las capas intermedias se configuran con `return_sequences=True` para propagar la secuencia temporal, mientras que la capa final resume el contexto global de la oración en un único vector denso (`return_sequences=False`).
- **Mecanismos de Regularización Avanzada:**
  - **SpatialDropout1D (0.2):** Aplicado entre capas recurrentes. A diferencia del *dropout* estándar, este apaga dimensiones completas del vector en toda la secuencia. Su uso es crítico para mantener la compatibilidad con los núcleos **CuDNN** de la GPU, permitiendo un entrenamiento eficiente sin renunciar a la regularización.
  - **Regularización L2 (0.001):** Integrada en la primera capa densa para penalizar pesos excesivos y favorecer patrones generales.

  - **Dropout Estándar (0.5):** Aplicado antes de la decisión final para forzar una clasificación robusta mediante la desactivación aleatoria del 50 % de las neuronas.

- **Cabecera de Clasificación:** La información procesada pasa por una capa `Dense` de 64 unidades con activación **ReLU** para capturar combinaciones no lineales. Finalmente, una capa de salida con activación **softmax** genera una distribución de probabilidad sobre las `num_clases`.

Para el entrenamiento, se emplea el optimizador **Adam** y la función de pérdida `categorical_crossentropy` mediante Keras [6] sobre TensorFlow [5], estándar en problemas de clasificación multiclase con etiquetas en formato *one-hot*.

4.4 Callbacks de entrenamiento

La función `definir_callbacks` configura dos *callbacks* con paciencias distintas para actuar en dos niveles:

- **EarlyStopping** (`patience=3`, `restore_best_weights=True`): detiene el entrenamiento si `val_loss` no mejora en 3 épocas y restaura los mejores pesos automáticamente.
- **ReduceLROnPlateau** (`factor=0.5`, `patience=1`, `min_lr=1e-6`): más impaciente — si `val_loss` se estanca una sola época reduce la tasa de aprendizaje a la mitad, permitiendo pasos más finos antes de que EarlyStopping aborte el entrenamiento.

4.5 Configuraciones evaluadas y resultados

Se evalúan **12 configuraciones** variando arquitectura (LSTM / GRU), número de unidades (16, 32, 64), número de capas (1, 2) y tasa de aprendizaje (0.0001, 0.001), con un máximo de `epochs = 8` y `batch_size = 64`. Todas las configuraciones utilizan el embedding `dim312_v2`.

Nombre	Arq.	Embed.	Unid.	Capas	LR	Val. Acc.
`LSTM_d16_1L_lr0001`	**LSTM**	**dim312**	**16**	**1**	**0.0001**	**89,12 %**
`LSTM_d16_1L_lr001`	LSTM	dim312	16	1	0.001	89,92 %
`LSTM_d32_1L_lr0001`	LSTM	dim312	32	1	0.0001	89,77 %
`LSTM_d32_1L_lr001`	LSTM	dim312	32	1	0.001	90,06 %
`LSTM_d64_1L_lr001`	LSTM	dim312	64	1	0.001	90,22 %
`LSTM_d32_2L_lr001`	LSTM	dim312	32	2	0.001	90,34 %
`LSTM_d128_1L_lr0001`	LSTM	dim752	128	1	0.0001	90,56 %
`LSTM_d64_1L_lr0001`	LSTM	dim752	64	1	0.0001	90,76 %
`LSTM_d64_1L_lr001`	LSTM	dim752	64	1	0.001	90,79 %
`GRU_d128_1L_lr0001`	GRU	dim752	128	1	0.0001	90,52 %
`GRU_d64_1L_lr0001`	GRU	dim752	64	1	0.0001	90,37 %
`GRU_d64_1L_lr001`	GRU	dim752	64	1	0.001	90,47 %
`GRU_d16_1L_lr0001`	GRU	dim312	16	1	0.0001	89,49 %
`GRU_d16_1L_lr001`	GRU	dim312	16	1	0.001	90,12 %
`GRU_d32_1L_lr0001`	**GRU**	**dim312**	**32**	**1**	**0.0001**	**89,76 %**
`GRU_d32_1L_lr001`	GRU	dim312	32	1	0.001	90,38 %
`GRU_d64_1L_lr001`	GRU	dim312	64	1	0.001	90,28 %
`GRU_d32_2L_lr001`	GRU	dim312	32	2	0.001	90,42 %

Cuadro 2: Configuraciones evaluadas y *val_accuracy* final.

Tras evaluar las diversas configuraciones presentadas en la Tabla 2, el modelo seleccionado como el más óptimo es `LSTM_d64_1L_lr001` (con *embeddings* `dim312`), el cual alcanza una *val_accuracy* del **90,22 %**.

Aunque ciertas configuraciones con `dim752` muestran precisiones ligeramente superiores en validación (superando el 90,7 %), se ha determinado que el uso de los *embeddings* más compactos (`dim312`) ofrece la mejor relación entre *accuracy* y generalización, minimizando la diferencia de pérdidas (*diff_loss*). La Figura 7 detalla la evolución del entrenamiento para las configuraciones probadas; estas gráficas, generadas mediante la función `graficar_resultados_entrenamiento`, permiten identificar visualmente cómo el modelo elegido logra un aprendizaje estable sin caer en el sobreajuste excesivo que caracteriza a las dimensiones superiores.

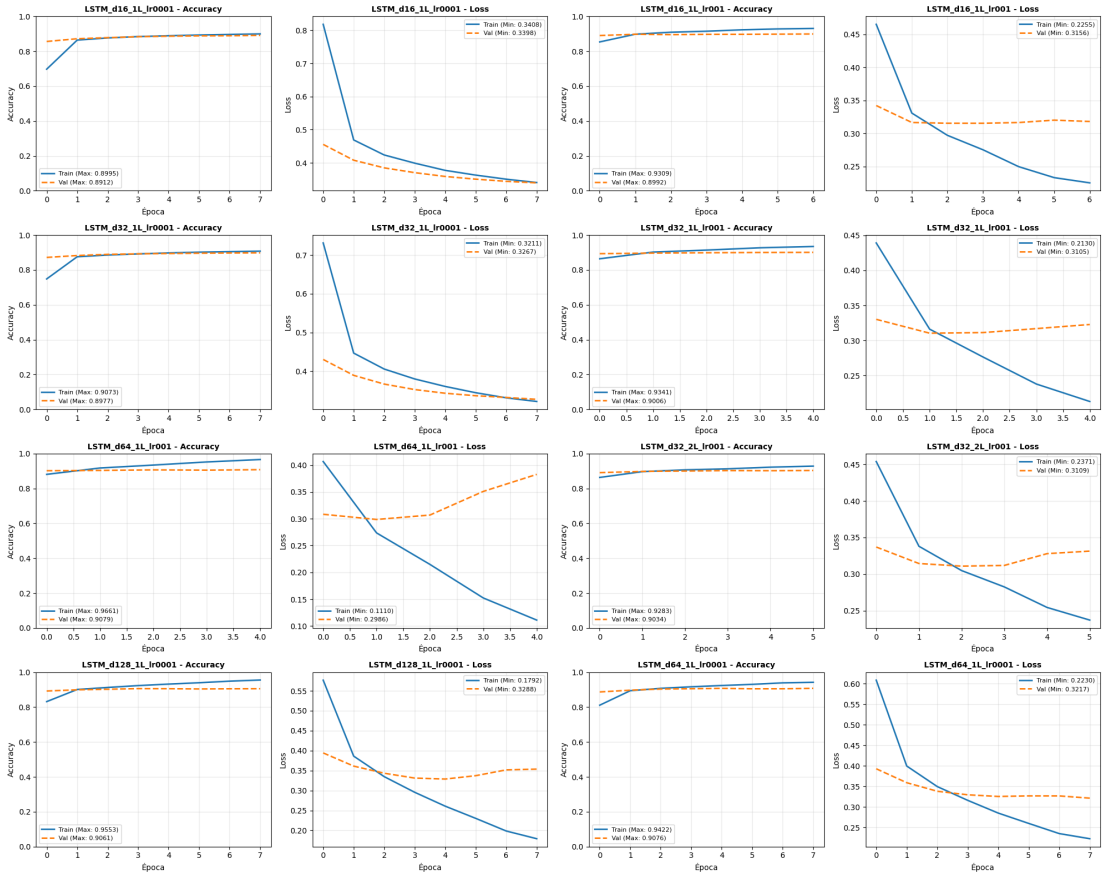

Figura 7: Evolución de *accuracy* y *loss* de las 18 configuraciones LSTM y GRU.

5 Evaluación y preguntas

5.1 Pregunta 1: ¿Ha alcanzado el modelo un entrenamiento óptimo?

Los modelos escogidos han tenido un entrenamiento óptimo, justificable a través de los siguientes puntos:

- **Generalización:** ambos modelos presentan una diferencia muy reducida entre la función de pérdida de entrenamiento y la de validación. El caso más destacado es `LSTM_d16_1L_lr0001`, con un `diff_loss` de tan solo 0,001, indicando que el fenómeno del *overfitting* prácticamente no se produce.
- **Convergencia estable:** ambos modelos muestran un comportamiento estable a lo largo de las épocas: el `val_loss` desciende de manera uniforme sin saltos bruscos, y el `accuracy` presenta una evolución consistente (Figura 7).

Para conseguir estos dos indicativos de un entrenamiento óptimo ha sido necesario emplear dentro de la arquitectura una regularización aplicada en diferentes capas y de diferentes formas: el uso de L2 y el uso de capas de `Dropout`.

5.2 Análisis de las curvas de entrenamiento

Del análisis de las curvas de la Figura 7 se extraen las siguientes conclusiones:

Los modelos con tasa de aprendizaje alta (`lr=0.001`) tienden a un sobreajuste más pronunciado y rápido. Por ejemplo, `LSTM_d16_1L_lr001` experimentó una diferencia final de *loss* (`diff_loss`) de 0,1054, activando rápidamente los *callbacks*. En cambio, los modelos con tasa de aprendizaje conservadora (`lr=0.0001`) presentan un aprendizaje mucho más estable: `LSTM_d16_1L_lr0001` logró un `diff_loss` de apenas 0,0010.

Por otro lado, los modelos más pequeños (16 o 32 unidades) muestran un comportamiento evolutivo más estable y convergente que los de mayor capacidad. Esto se demuestra empíricamente: el mejor modelo LSTM fue precisamente el de 16 unidades (`LSTM_d16_1L_lr0001`, `diff_loss=0.0010`), y el mejor GRU fue el de 32 unidades (`GRU_d32_1L_lr0001`, `diff_loss=0.0023`).

5.3 Pregunta 2: ¿Qué revela la matriz de confusión sobre el rendimiento?

Las Figuras 8 y 9 muestran las matrices de confusión del mejor modelo LSTM (`LSTM_d16_1L_lr0001`) y del mejor GRU (`GRU_d32_1L_lr0001`) sobre el conjunto de test estándar.

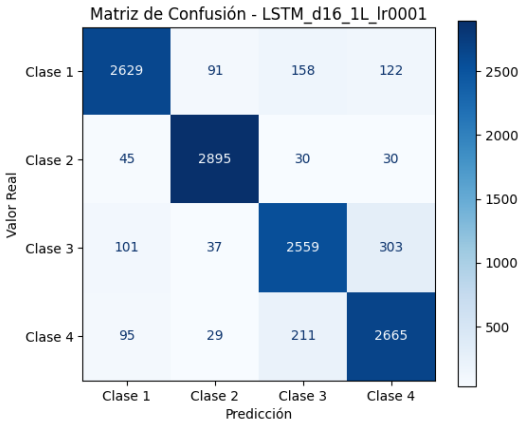

Figura 8: Matriz de confusión — mejor LSTM (`LSTM_d16_1L_lr0001`).

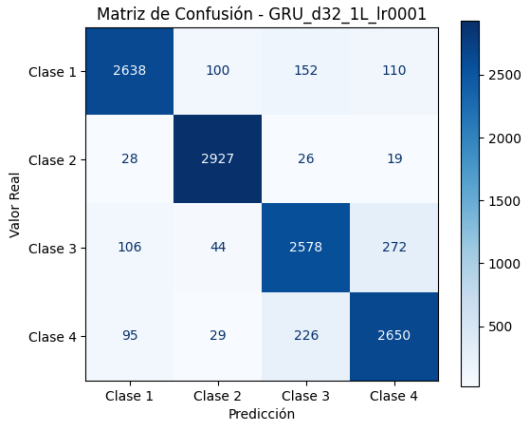

Figura 9: Matriz de confusión — mejor GRU (`GRU_d32_1L_lr0001`).

La matriz de confusión revela un rendimiento muy sólido y consistente de ambos modelos. Observamos una fuerte concentración de predicciones en la diagonal principal, indicando que la gran mayoría de las muestras han sido clasificadas correctamente. Los valores fuera de la diagonal son relativamente bajos y simétricos, indicando que el modelo no está excesivamente sesgado hacia ninguna clase. Se aprecia una ligera tendencia a la confusión entre las clases 1, 3 y 4, siendo la clase 2 la que menor tasa de fallo presenta.

En cuanto a la capacidad de generalización, el comportamiento de la matriz tanto sobre los datos de test como sobre el corpus personalizado (*custom*) corrobora que el modelo ha aprendido características semánticas subyacentes útiles y no simplemente memorizado los datos. Las ligeras confusiones están justificadas por el solapamiento natural del vocabulario entre temáticas afines (por ejemplo, noticias empresariales frente a tecnología).

Ambos modelos comparten un comportamiento sobre las predicciones bastante similar, lo que indica que, a pesar de las diferencias en dimensión de embedding, número de neuronas y otros parámetros, el entrenamiento converge a soluciones equivalentes. Esto puede ser indicativo de que la naturaleza del corpus introduce un ligero sesgo en ciertos documentos de clases específicas, lo que genera una leve tendencia a la confusión entre categorías afines.

Para evaluar la capacidad de generalización a datos reales, se construye un corpus de 20 noticias extraídas de fuentes ajenas al corpus de entrenamiento (5 por categoría), aplicando el mismo preprocesamiento que al corpus original. Las Figuras 10 y 11 muestran los resultados.

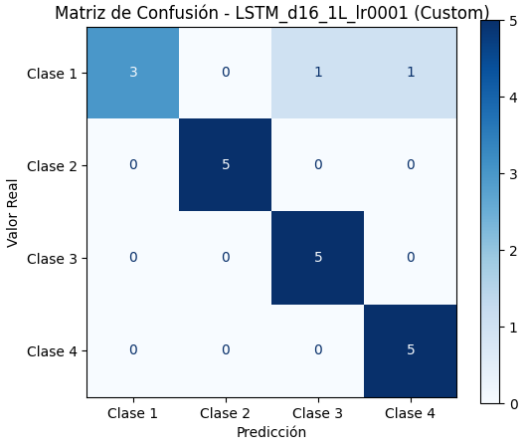

Figura 10: Matriz de confusión — mejor LSTM sobre corpus personalizado (20 noticias).

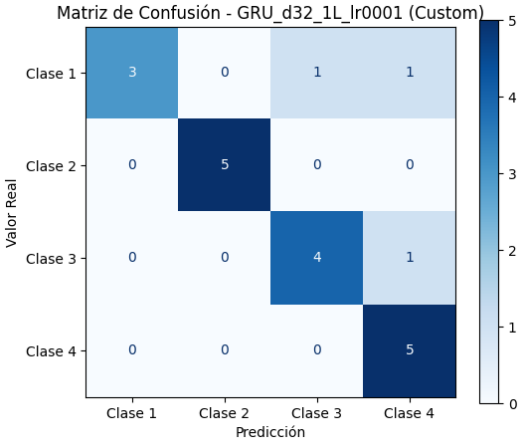

Figura 11: Matriz de confusión — mejor GRU sobre corpus personalizado (20 noticias).

Una vez procesados los datos del corpus de evaluación, el análisis de las predicciones permite discernir entre ambos modelos cuál es más adecuado para un potencial proceso de inferencia. El comportamiento sobre el corpus personalizado corrobora que los modelos han aprendido características semánticas subyacentes útiles y no han memorizado los datos de entrenamiento, confirmando su capacidad de generalización a texto real.

5.4 Pregunta 3: ¿Qué diferencia de precisión se observa al cambiar de embedding?

Al comparar el rendimiento entre las incrustaciones de menor dimensión (`dim312_v2`) y las de mayor dimensión (`dim752_v3`), se observan los siguientes puntos clave:

- **Impacto en la precisión:** El aumento de dimensionalidad no supuso una mejora significativa ni estable en la precisión de validación.
- **Sobreajuste (*overfitting*):** Se detectó que una mayor dimensión incrementó la tendencia del modelo al sobreajuste.
- **Eficiencia:** Los modelos que lograron la mejor relación entre *accuracy* y menor diferencia de pérdidas (`diff_loss`) fueron aquellos entrenados con los *embeddings* más compactos (`dim312_v2`).

Conclusión técnica:

Esto se debe a que una mayor cantidad de dimensiones introduce una cantidad mucho mayor de parámetros en el modelo. Esto facilita que la red memorice ruido o características muy específicas del conjunto de entrenamiento, en lugar de aprender representaciones lingüísticas generalizables a datos no vistos.

5.5 Pregunta 4: ¿LSTM y GRU reaccionaron de forma diferente a los embeddings?

En términos generales, ambas arquitecturas (LSTM y GRU) reaccionaron de forma muy similar a los dos tipos de incrustación. Ninguna de las dos obtuvo un beneficio claro de utilizar los embeddings de mayor dimensionalidad (`dim752_v3`). Por el contrario, tanto LSTM como GRU sufrieron un mayor sobreajuste al emplear las incrustaciones de 752 dimensiones, debido al drástico aumento en la cantidad de parámetros a entrenar. Como resultado, los modelos más óptimos y estables para ambas arquitecturas se obtuvieron utilizando las incrustaciones de menor dimensión (`dim312_v2`), demostrando que, para este corpus y problema específico, una representación más compacta es preferible independientemente del tipo de celda recurrente utilizada.

6 Conclusiones

Este trabajo cubre el proceso completo de desarrollo de un sistema de clasificación de textos mediante aprendizaje profundo, abarcando desde la preparación del dataset hasta la comparativa final de modelos.

En el **Ejercicio 1**, se ha comprobado que el algoritmo Skip-Gram [2] con *negative sampling* genera representaciones de calidad incluso con un corpus limitado. Los seis entrenamientos realizados muestran una convergencia clara, estabilizando la pérdida en el rango de $0,35$ a $0,38$. Los datos sugieren que aumentar la dimensionalidad y la ventana de contexto ayuda a reducir el error de forma consistente. Gracias a un vocabulario de 26,721 palabras, los clasificadores del siguiente ejercicio lograron superar el 90 % de *accuracy* utilizando únicamente estos *embeddings* generados desde cero.

Respecto al **Ejercicio 2**, los resultados indican que no hay una ventaja clara entre usar LSTM o GRU para este tipo de textos cortos; la diferencia de rendimiento entre ambas fue de apenas el 0,15 %. Para lograr una convergencia rápida y evitar el sobreajuste, fue fundamental combinar `SpatialDropout1D` con regularización L2 y una gestión dinámica del *learning rate*. El análisis de la matriz de confusión revela que la mayoría de errores ocurren entre las categorías de *World* y *Business*, un solapamiento lógico dada la terminología política y económica que comparten.

De cara a ampliar este estudio, se plantean tres vías:

1. Implementar modelos basados en atención (*Transformers*) para intentar resolver la ambigüedad en las categorías más próximas; y
2. Contrastar el rendimiento de los vectores propios frente a pesos preentrenados de GloVe o FastText [2].

Bibliografía

Referencias

[1] Zhang et al. (2015) — AG News / clasificación de texto.

[2] Mikolov et al. (2013) — Word2Vec / Skip-Gram.

[3] Hochreiter & Schmidhuber (1997) — LSTM.

[4] Cho et al. (2014) — GRU.

[5] Abadi et al. (2015) — TensorFlow. `https://www.tensorflow.org`

[6] Chollet et al. (2015) — Keras. `https://keras.io`